

EECS 348 Assignment 3

Author: Peter Lauman

Course: EECS 348 Software Engineering 1

Assignment: Assignment 3

Date: October 1, 2026

1. GenAI Programs and Access

GenAI 1: Google Gemini

I used Google Gemini as the first GenAI program. I accessed Gemini through the Gemini website using a web browser.

GenAI 2: Claude

I used Claude as the second GenAI program. I accessed Claude through the Claude website using a web browser.

2. Prompt Used

I used the same prompt for both Gemini and Claude:

Generate a C++ program using objects, not functions, that prioritizes emails for a busy company CEO. Implement a MaxHeap from scratch using a list-based implementation. Do not use any pre-existing heap modules.

The program must read commands from a test file. The commands are EMAIL, NEXT, READ, and COUNT.

EMAIL has the format:

EMAIL sender category,subject line,date

Sender priority from highest to lowest is:

Boss

Subordinate

Peer

ImportantPerson

OtherPerson

If two emails have the same sender category, the newest email must have higher priority. Dates are formatted MM-DD-YYYY.

NEXT displays the highest-priority unread email in this format:

Next email:

Sender: [sender]

Subject: [subject]

Date: [date]

NEXT must not remove the email, so two NEXT commands in a row display the same email.

READ removes the highest-priority email. READ must also work when there was no preceding NEXT. Two READ commands in a row should remove the two highest-priority emails.

COUNT displays:

There are [number] emails to read.

NEXT and READ must safely handle an empty inbox.

Use C++ classes/objects rather than a function-based design. Create the MaxHeap yourself using a list-based implementation and do not use `std::priority_queue` or any existing heap implementation. The program should accept the test-file name as a command-line argument.

3. Generated Code

Both generated code are in the folder for size purposes

4. Analysis of Gemini and Claude Code

Correctness

Gemini

The Gemini program correctly implements the main requirements of the assignment. It creates Email, MaxHeap, and EmailSystem classes and uses a vector to store the elements of the MaxHeap.

The program correctly assigns the following sender priorities:

Boss > Subordinate > Peer > ImportantPerson > OtherPerson.

It also converts dates from MM-DD-YYYY into an integer arranged as YYYYMMDD. This allows newer dates to have a higher priority when two emails have the same sender category. The MaxHeap implementation correctly moves higher-priority emails toward the root. NEXT obtains the root without removing it, while READ removes the root. One weakness of the Gemini implementation is that getSenderPriority() and parseDate() are global helper functions rather than functionality contained inside an object. Because this assignment specifically emphasizes using objects rather than functions, moving this logic inside a class would provide a more object-oriented design. Gemini also prints additional messages when NEXT or READ is performed on an empty inbox. The assignment says the program must be able to handle those situations, but it does not specify that those additional messages should be printed.

Claude

Claude also correctly implements the required priority system and creates its own MaxHeap without using std::priority_queue. Claude divides the program into four main classes: Email stores individual email information and comparison logic. MaxHeap implements the priority queue. Inbox manages the CEO's unread emails. CommandProcessor reads and processes commands from the input file. Claude correctly handles sender priority and date priority. It also correctly allows NEXT to display an email without removing it and allows READ to remove the highest-priority email without requiring a previous NEXT. Claude's design places the sender priority and date conversion logic inside the Inbox class rather than using global helper functions. Overall, Claude's organization provides a better starting point for an object oriented solution.

5. Execution Time Analysis

Both programs have very similar execution-time complexity because both implement a binary MaxHeap.

EMAIL — $O(\log n)$

When an email is inserted, it is first placed at the end of the heap. It may then move upward through the heap until it reaches its proper position.

Because a binary heap has a height of approximately $\log n$, insertion takes:

$O(\log n)$

READ — $O(\log n)$

READ removes the highest-priority email at the root.

The last email is moved to the root and may need to move downward through the heap.

Therefore:

$O(\log n)$

NEXT — $O(1)$

The highest-priority email is always located at index 0.

The program can access it directly without searching.

Therefore:

$O(1)$

COUNT — $O(1)$

The number of elements can be obtained directly from the vector.

Therefore:

$O(1)$

Neither Gemini nor Claude has a significant asymptotic execution-time advantage.

6. Space Complexity

Both programs store each unread email once inside a vector. If there are n unread emails, the heap therefore requires: $O(n)$ space. Both implementations perform their heap operations iteratively rather than recursively, so they do not require an additional recursive call stack. The overall space complexity for both programs is: $O(n)$

7. Maintainability

The Claude program is more maintainable because responsibilities are divided more clearly between classes.

The Email class represents and compares emails.

The MaxHeap class manages the heap.

The Inbox class manages CEO inbox operations.

The CommandProcessor class handles the input file.

Gemini's program is somewhat shorter, but some important email-related logic is contained in global functions.

Claude's separation of responsibilities makes it easier to modify one part of the program without changing unrelated sections.

For example, changes to the heap implementation can primarily remain inside MaxHeap, while changes to file processing can remain inside CommandProcessor.

8. Selected Program

I selected the Claude program as the basis for my final Assignment 3 program.

Both programs have essentially the same execution-time and space complexity, so performance was not the main reason for selecting Claude.

Claude was selected because its program has a clearer object-oriented structure. The responsibilities of storing emails, maintaining the MaxHeap, managing the inbox, and processing commands are separated into different classes.

This makes the program easier to understand, modify, debug, and maintain.

9. Improvements Made to the Claude Program

Correctness

I simplified the command parsing so that it follows the exact format guaranteed by the assignment.

The assignment states that an email subject may contain spaces but will not contain commas. Therefore, the final program only needs the first and second commas to separate sender, subject, and date.

I also changed the handling of empty NEXT and READ commands so that the program safely performs no operation instead of producing additional output that is not specified by the assignment.

NEXT still leaves the highest-priority email in the heap.

READ always removes the highest-priority email even if NEXT was never called.

This also means two consecutive NEXT commands display the same email, while two consecutive READ commands remove two different emails as required.

Execution Time

The binary MaxHeap was retained because it already provides efficient operations.

The final complexities remain:

EMAIL: $O(\log n)$

READ: $O(\log n)$

NEXT: $O(1)$

COUNT: $O(1)$

The entire inbox never needs to be sorted after every operation.

Space Complexity

The final implementation continues to require $O(n)$ space.

Each unread email is stored once in the vector that represents the MaxHeap.

The heap algorithms remain iterative, avoiding additional recursive stack usage.

Maintainability

I kept the object-oriented structure of Claude's solution but reorganized some responsibilities.

The final program contains:

Email — stores email data and determines email priority.

MaxHeap — implements the list-based binary MaxHeap.

Inbox — implements EMAIL, NEXT, READ, and COUNT behavior.

CommandProcessor — reads and interprets the test file.

I also added detailed comments explaining the purpose of the classes, major blocks, and individual operations.

The source comments identify Claude and Gemini as GenAI sources and identify Claude's program as the baseline for the final implementation.